

TEXT ANALYTICS

Representing Text and Information Retrieval

Francesco Spinnato

* francesco.spinnato@unipi.it

University of Pisa



From Text to Numerical Representations

From Linguistic Enrichment to Features

Most data mining and machine learning algorithms work on numbers, not on strings. The simplest bridge is to decide **in advance** which properties of a text are worth measuring, and then to measure them.

Linguistic enrichment

Adds **structure and linguistic knowledge** to raw text.

Examples: POS tags, lemmas, morphological features, syntactic dependencies, named entities, ...

The output is still **symbolic**, not numerical yet.

Feature extraction

Selects and quantifies relevant information from the enriched text.

Examples: frequency of specific POS tags or entities, average sentence length, number of emojis or hashtags

The output is **quantified evidence** (counts, ratios, frequencies).

Representing Text with Linguistic Features

The goal is to convert a corpus into a design matrix $\mathbf{X} \in \mathbb{R}^{N \times |F|}$.

We fix a set of **features** $F = \{f_1, \dots, f_{|F|}\}$ and turn each document d into a vector $\mathbf{x}_d$, where x_{df} is the value of feature f measured on d .

Kind of feature	Measured on d	Requires
Surface	length, average word length, punctuation counts	the raw string
Lexical	frequency of a chosen word or phrase	tokenization
Grammatical	how many nouns, verbs, adjectives	POS tagging
Entity	how many people, places, organizations	NER

When rows represent documents and columns represent terms, the design matrix is called a **document-term matrix**.

Encoding Feature Values

After extraction, each feature must be encoded as a numeric value so that texts are comparable. Every column has its own meaning, unit and scale.

- **Boolean features**, encode presence/absence as 1/0 (e.g. presence of a question mark, or presence of an Entity).
- **Categorical features**, encode a category as a (multi-hot) boolean vector, with as many entries as the number of categories, e.g., if a document contains entities A and B but not C, the document is [1, 1, 0]
- **Count features**, raw frequencies of linguistic units (e.g. number of nouns, verbs, emojis),
- **Ratio / relative-frequency features**, normalized counts by a denominator (tokens, sentences, paragraphs) to control for length: e.g. n. of adjectives/n. of tokens, n. of capital letters/n. of characters.

Encoding Feature Values: Encoding and Scaling

A linguistic features design matrix mixes quantities of different kinds, and they cannot all be treated alike.

You should **apply different treatments to different column groups**, rather than scaling the whole matrix indiscriminately. E.g.:

- **counts** (number of entities, number of adjectives) are unbounded and skewed, so they are usually **log-transformed** or **scaled** (e.g., min-max or z-score normalization)
- **ratios** (proportion of nouns) already live in $[0, 1]$ and should be **left alone**
- **categorical labels** (dominant POS tag, genre, language) carry no order, so they are **one-hot encoded**, one column per value

The Same Features, Two Very Different Vectors

For example, take the documents:

d_1 *"Thou art more lovely and more temperate."*

d_2 *"Yo, I'm up before six, I don't do what I'm told."*

Stacking one row per document gives the **design matrix** X , of size $N \times |F|$ for a corpus of N documents, in this case 2.

	chars	words	avg w. len	apostrophes	1st person	adj.	entities
$\mathbf{x}_{d_1}$	40	7	4.7	0	0	2	0
$\mathbf{x}_{d_2}$	48	11	2.9	3	3	0	0

Strengths and Limitations

Handcrafted features are interpretable because each feature represents a property that we can name.

- This allows documents to be compared according to the properties encoded by the selected features.

If we select appropriate features, we can use, for example, Euclidean distance to find similar documents, such as song lyrics or poems, or to discriminate between different types or authors of documents, such as rap and classical lyrics or texts written by Dante and Boccaccio.

- Handcrafted features can also be used as input for downstream tasks such as classification, clustering, authorship attribution, and information retrieval.
- However, the feature set may need to be redesigned for each task, and its design requires domain knowledge.

Bag of Words and Weighting Schemes

From Handcrafted Features to Bag of Words

Instead of handcrafting features, can we obtain a general-purpose representation directly from the text?

A simple solution is to use the words themselves as features:

- Each distinct word defines a feature.
- Documents are represented according to the words they contain.
- Different weighting schemes determine the importance of each word.

This leads to the **Bag-of-Words model**.

One-Hot Representation of Words

The most common starting representation for words is one-hot encoding. Each distinct word type is assigned **its own dimension** in the feature space $\mathbb{R}^{|V|}$, where $|V|$ is the size of the vocabulary.

Here, the features are the word types, so $|F| = |V|$.

A one-hot vector encodes a single word:

$$\mathbf{e}_f = [0, \dots, 1, \dots, 0]$$

where 1 marks the coordinate corresponding to word feature f .

One-Hot Representation of Words

Take a vocabulary of $|V| = 8$ types, held in a fixed order so that coordinate 3 always means the same thing:

$V = (\text{a, bad, game, good, lose, play, the, you})$

$$\mathbf{e}_{\text{game}} = [0, 0, 1, 0, 0, 0, 0, 0] \quad \mathbf{e}_{\text{good}} = [0, 0, 0, 1, 0, 0, 0, 0]$$

Bag-of-Words: From Words to Documents

A one-hot vector stands for a single word. A **document** contains many, so its vector carries one coordinate for every word type in the vocabulary, whether or not the document uses it.

In a **Bag-of-Words (BoW)** a document is described by the vocabulary terms it contains, ignoring their order.

Write a document as a sequence of n tokens, $d = (w_1, \dots, w_n)$. The simplest BoW representation is the **term-count** matrix, in which a document is just the sum of its words' one-hot vectors:

$$\mathbf{x}_d = \sum_{i=1}^n \mathbf{e}_{w_i}$$

Addition is commutative, so any reordering of d gives the same vector. That is what *bag* means.

Bag-of-Words: A Worked Example

$V = (a, bad, game, good, lose, play, the, you)$

$d = \text{“you play a good game, a good game”}$

$$\begin{aligned}\mathbf{x}_d &= \mathbf{e}_{you} + \mathbf{e}_{play} + \mathbf{e}_a + \mathbf{e}_{good} + \mathbf{e}_{game} + \mathbf{e}_a + \mathbf{e}_{good} + \mathbf{e}_{game} \\ &= \left[\underbrace{2}_a, \underbrace{0}_{bad}, \underbrace{2}_{game}, \underbrace{2}_{good}, \underbrace{0}_{lose}, \underbrace{1}_{play}, \underbrace{0}_{the}, \underbrace{1}_{you} \right]\end{aligned}$$

- Coordinates 2, 5 and 7 are 0: *bad*, *lose* and *the* do not occur in d .
- With a realistic $|V|$ almost every coordinate is 0, giving a **sparse** representation.

Bag of Words

Fixed-length representation

The BoW representation yields a fixed-size vector independent of the document length, making it easier to handle for most ML algorithms.

Document	Text	a	bad	game	good	lose	play	the	you
d_1	“you play a good game, a good game”	2	0	2	2	0	1	0	1
d_2	“you lose the game”	0	0	1	0	1	0	1	1
d_3	“a bad game”	1	1	1	0	0	0	0	0
d_4	“you play the good game”	0	0	1	1	0	1	1	1

Bag of Words and n-grams

Loss of word order

BoW disregards the sequence of words, similarly to unigram models. For example, although the following two sentences have **opposite meanings**, their Bag-of-Words representations are **identical**.

$$d_1 = \text{“I won, and thus you lose”} \quad d_2 = \text{“I lose, and thus you won”}$$

Word n -grams can partially recover local ordering information.

Document	i won	won and	i lose	lose and	and thus	thus you	you lose	you won
d_1	1	1	0	0	1	1	1	0
d_2	0	0	1	1	1	1	0	1

Character n -grams can also be useful for handling **typos** and **morphological variants**.

Weighting Schemes in Text Representation

In text representation, each term (word, lemma or n -gram) can be assigned a **weight** reflecting its relevance within a text and across the corpus.

Two dimensions of weighting

- **Local weighting:** importance of a term *within a specific text*.
- **Global weighting:** importance of a term *across all texts* in the corpus.

Main schemes

- **TF** (term frequency), local weight
- **IDF** (inverse document frequency), global weight
- **TF-IDF**, combines local and global weights

Term Frequency (TF)

The **Term-Count** representation assigns larger values to words that occur frequently. Because longer documents generally contain more words, they also tend to produce vectors with larger magnitudes. Alternatives include:

Term Frequency (TF): divides each term count by the document length:

$$\mathbf{x}_d = \frac{1}{n_d} \sum_{i=1}^{n_d} \mathbf{e}_{w_i},$$

where n_d is the number of word tokens in document d

e.g. $d = \text{"climate change change policy"} \Rightarrow \text{TF}(\text{change}) = 2/4 = 0.5$

Inverse Document Frequency (IDF)

TF measures how frequently a term occurs **locally**, within a document. However, terms that occur in many documents, such as *the*, *and*, *of* and *in*, usually carry little discriminative information.

Inverse Document Frequency (IDF) measures how rare or discriminative a term is across the corpus:

$$\text{IDF}(w) = \log_{10} \frac{N}{\text{DF}(w)},$$

where N is the total number of documents and $\text{DF}(w)$ is the number of documents containing term w .

Inverse Document Frequency (IDF)

$$\text{IDF}(w) = \log \frac{N}{\text{DF}(w)},$$

Intuitively:

- the term inside the log, $N / \text{DF}(w)$, means that one out of every $N / \text{DF}(w)$ documents contains the word w , on average.
- The logarithm dampens the influence of extremely rare words.

Thus IDF is a sort of **global rarity** score:

- A term appearing in few documents has a high IDF (rare word).
- A term appearing in many documents has a low IDF (common word).

Term Frequency - Inverse Document Frequency is defined as

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w)$$

TF-IDF is high when a term is frequent in **one** document but rare **overall**.

- TF-IDF identifies words that are prominent in one document but uncommon across the corpus.
- It is commonly used for document classification, information retrieval, and topic detection.

Example: Computing TF, IDF and TF-IDF

d_1 : The car is driven on the road d_2 : The truck is driven on the highway.

1. Create a term frequency table.

Each sentence has 7 tokens

Term	$TC(w, d_1)$	$TF(w, d_1)$	$TC(w, d_2)$	$TF(w, d_2)$
the	2	0.286	2	0.286
car	1	0.143	0	0
is	1	0.143	1	0.143
driven	1	0.143	1	0.143
on	1	0.143	1	0.143
road	1	0.143	0	0
truck	0	0	1	0.143
highway	0	0	1	0.143

Example: Computing TF, IDF and TF-IDF

d_1 : The car is driven on the road d_2 : The truck is driven on the highway.

Term	DF(w)	IDF(w) = $\log(2/\text{DF}(w))$
the	2	$\log(2/2) = 0.000$
car	1	$\log(2/1) = 0.301$
is	2	$\log(2/2) = 0.000$
driven	2	$\log(2/2) = 0.000$
on	2	$\log(2/2) = 0.000$
road	1	$\log(2/1) = 0.301$
truck	1	$\log(2/1) = 0.301$
highway	1	$\log(2/1) = 0.301$

2. Calculate inverse document frequency.

where $N = 2$

Example: Computing TF, IDF and TF-IDF

3. Combine term frequency and inverse document frequency:

$$\text{TFIDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w)$$

Term	TF-IDF (d_1)	TF-IDF (d_2)
the	$0.286 \times 0.000 = 0.000$	$0.286 \times 0.000 = 0.000$
car	$0.143 \times 0.301 = 0.043$	$0.000 \times 0.301 = 0.000$
is	$0.143 \times 0.000 = 0.000$	$0.143 \times 0.000 = 0.000$
driven	$0.143 \times 0.000 = 0.000$	$0.143 \times 0.000 = 0.000$
on	$0.143 \times 0.000 = 0.000$	$0.143 \times 0.000 = 0.000$
road	$0.143 \times 0.301 = 0.043$	$0.000 \times 0.301 = 0.000$
truck	$0.000 \times 0.301 = 0.000$	$0.143 \times 0.301 = 0.043$
highway	$0.000 \times 0.301 = 0.000$	$0.143 \times 0.301 = 0.043$

Example: What TF-IDF Tells Us

- Words like "*the*", "*is*", "*driven*", "*on*" appear in **both** documents, and therefore have $IDF = 0$: uninformative, common language.
- Words like "*car*", "*road*", "*truck*", "*highway*" appear in only **one** document, so they get a high IDF, these are more **discriminative**.
- TF-IDF assigns higher weight to document-specific content words, and downweights frequent functional words.

Vector Space Models

With document-term matrices we are moving toward a **geometric interpretation** of text.

- Each document is represented as a vector $\mathbf{x}_d$ in a high-dimensional space $\mathbb{R}^{|F|}$, where $|F|$ is the number of extracted features (e.g. terms, linguistic features), or the number of word types in the BoW.
- The "position" of a document vector reflects its content: documents with similar feature patterns occupy nearby regions of the space.
- The **Vector Space Model (VSM)** treats similarity between texts as *geometric proximity* between their corresponding vectors.

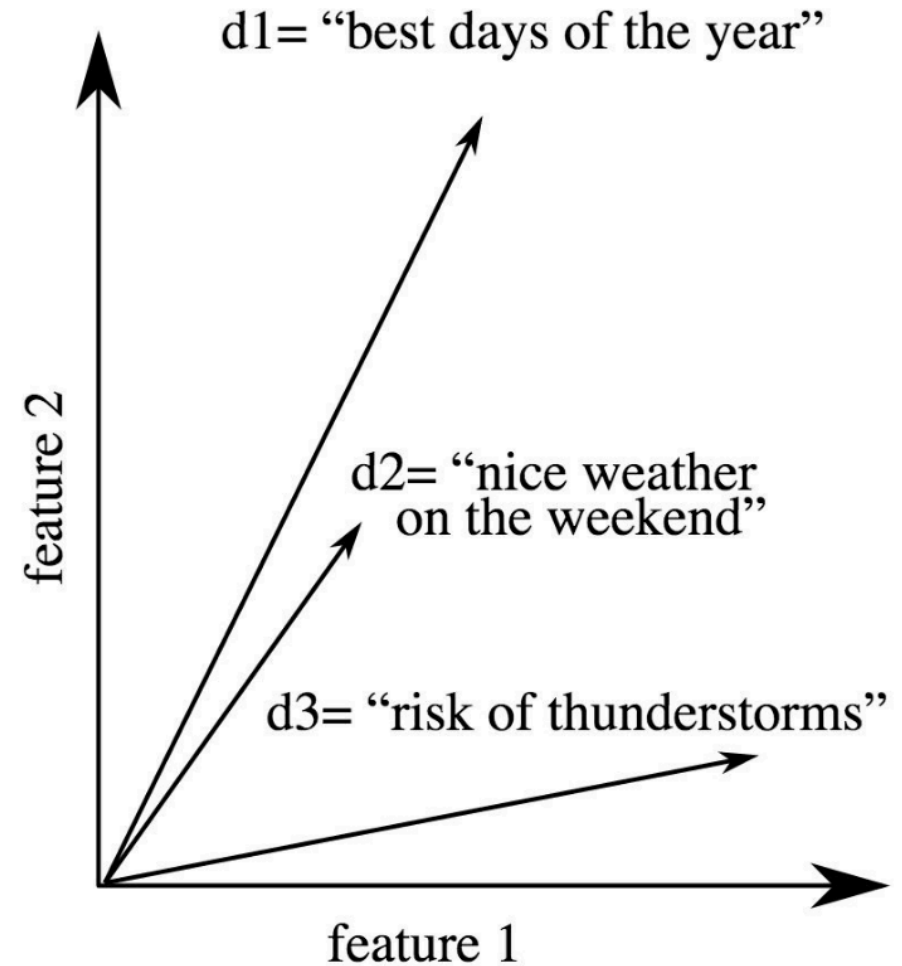
The VSM enables a unified view of text comparison, retrieval and clustering through linear algebra and geometry, and provides the foundation for later semantic models.

Similarity Between Texts

Geometry of Document Vectors

Each document is represented by a vector $\mathbf{x}_d \in \mathbb{R}^{|V|}$. Document vectors are **sparse** and **high-dimensional**; most coordinates are zero.

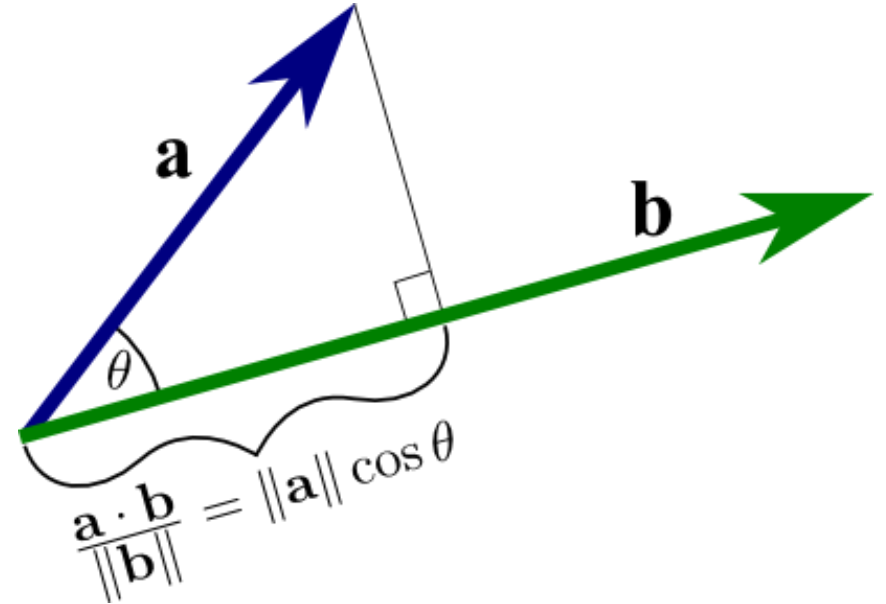
- Geometrically, the vector can be seen either
 - as an **arrow** from the origin, or
 - as the **point** marking its tip in feature space.
- Documents with **similar feature patterns** have vectors pointing in **similar directions**.
- The **angle** between vectors captures topical similarity independently of document length.



Dot Product

Most similarity functions derive from the **dot product** (inner product) between vectors:

$$\begin{aligned}\mathbf{a} \cdot \mathbf{b} &= \sum_{i=1}^n a_i b_i = a_1 b_1 + \cdots + a_n b_n \\ &= \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta\end{aligned}$$



The geometrical intuition is that the dot product is the length of the projection of **a** onto **b**, scaled by the length of **b**. It depends on:

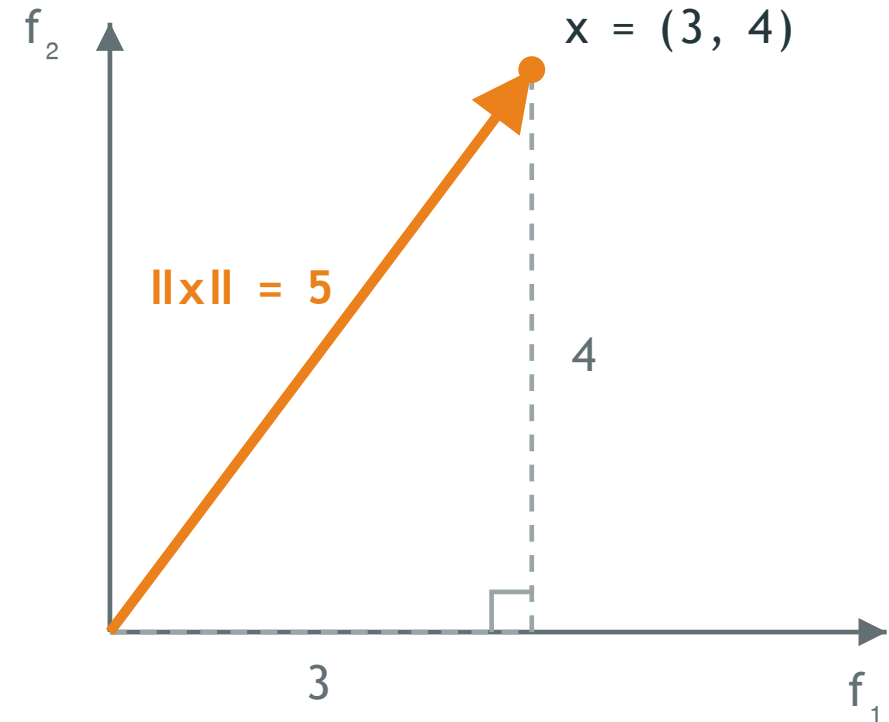
- **orientation**: the angle θ between **a** and **b**
- **length**: the magnitude of **a** and **b** : $\|\mathbf{a}\|, \|\mathbf{b}\|$

The Norm

The **norm**, or magnitude, of a vector is its length, that is, the distance from the origin to its tip:

$$\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x} \cdot \mathbf{x}}$$

- For a **document** vector the norm grows with the number of words: repeat a text twice and every count doubles, so the norm doubles while the topic has not changed at all.



In two dimensions this is just Pythagoras, and the same formula carries over unchanged to n dimensions, where we can no longer draw it.

Dot Product

The dot product between two vectors measures how much they are **aligned**:

- it is a measure of *overlap*, and grows when the two vectors have large values on the same dimensions
- for documents: two texts are similar if they share heavily weighted terms
- however, the dot product is also affected by **vector magnitude**

d_1 : play the game to win d_2 : play the game to win play the game to win

	game	play	the	to	win	$\mathbf{x}_{d_1} \cdot \mathbf{x}$
$\mathbf{x}_{d_1}$	1	1	1	1	1	5
$\mathbf{x}_{d_2}$	2	2	2	2	2	10

So d_1 comes out **twice** as similar to a doubled copy of itself as to itself.

Cosine Similarity

We are often interested in **directional** similarity, not in vector magnitude. Dividing by the product of the vector lengths removes length effects:

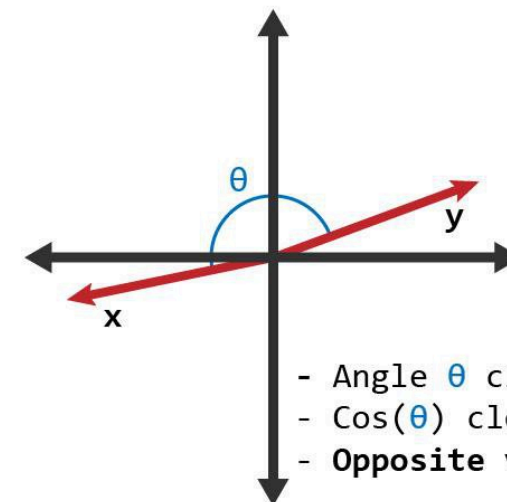
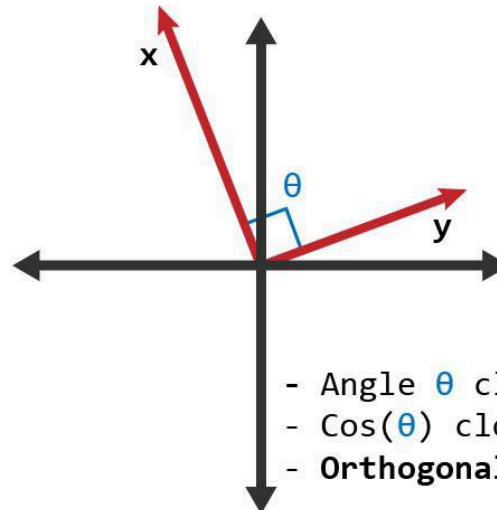
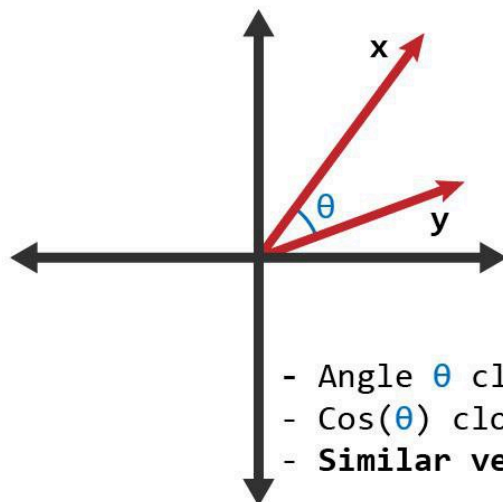
$$\text{cosine}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\|\mathbf{a}\| \|\mathbf{b}\| \cos \theta}{\|\mathbf{a}\| \|\mathbf{b}\|} = \cos \theta$$

Cosine similarity is **length-invariant**, making it ideal for comparing frequency-based or weighted vectors.

Cosine Similarity

Cosine similarity is a normalized dot product that depends only on the angle between vectors:

- $\cos(\theta) = 1$: identical direction (maximal similarity)
- $\cos(\theta) = 0$: orthogonal (unrelated)
- $\cos(\theta) = -1$: opposite orientation (only possible when vectors have negative entries); for count and TF-IDF vectors cosine lies in $[0, 1]$.



Cosine Similarity - Example

Let's take d_1 , d_2 , and add a genuinely shorter document d_3 : *"play the game"*.

	game	play	the	to	win	$\mathbf{x}_{d_1} \cdot \mathbf{x}$	$\ \mathbf{x}\ $	$\text{cosine}(\mathbf{x}_{d_1}, \mathbf{x})$
$\mathbf{x}_{d_1}$	1	1	1	1	1	5	$\sqrt{5}$	1.00
$\mathbf{x}_{d_2}$	2	2	2	2	2	10	$2\sqrt{5}$	1.00
$\mathbf{x}_{d_3}$	1	1	1	0	0	3	$\sqrt{3}$	0.77

$$\text{cosine}(\mathbf{x}_{d_1}, \mathbf{x}_{d_2}) = \frac{10}{\sqrt{5} \cdot 2\sqrt{5}} = \frac{10}{10} = 1 \quad \text{cosine}(\mathbf{x}_{d_1}, \mathbf{x}_{d_3}) = \frac{3}{\sqrt{5} \cdot \sqrt{3}} = \frac{3}{\sqrt{15}} \approx 0.77$$

- Doubling the text doubles the numerator **and** the denominator, so the two cancel and d_2 is correctly recognized as the same document.
- Meanwhile d_3 still scores below 1: cosine removes length without flattening real differences.

What Similarity Is For

We can now put a single number on how close two texts are. That one operation is what makes the vector space worth building.

If the text we start from is a user's **question**, then "*find the closest documents*" is exactly what a search engine does, and every piece needed to build one is already on the previous slides:

- a vocabulary,
- TF-IDF weights,
- and the cosine.

This is **retrieval**: given one text, find the documents closest to it in the collection.

Information Retrieval

The Retrieval Problem

Ad hoc retrieval: a user has an *information need*, expresses it as a **query**, and the system returns a ranked list of documents from a **collection**.

- The **collection** is the corpus C , the N documents we have indexed.
- The **query** q is a short piece of text, typically two or three words.
- The output is not a yes/no answer but a **ranking**: the documents most likely to satisfy the need, best first.

Everything we need is already built. A query is just a very short document, so we can put it in the same space $\mathbb{R}^{|V|}$, weight it the same way, and compare it with the tool we already have.

Ranking by Cosine

Represent the query as a vector $\mathbf{q}$ over the same vocabulary, then score every document by its cosine with the query:

$$\text{score}(q, d) = \text{cosine}(\mathbf{q}, \mathbf{x}_d) = \frac{\mathbf{q} \cdot \mathbf{x}_d}{\|\mathbf{q}\| \|\mathbf{x}_d\|}$$

and return the documents in decreasing order of score.

- Only the **query terms** contribute to $\mathbf{q} \cdot \mathbf{x}_d$: every other coordinate of $\mathbf{q}$ is 0. This will matter a great deal in a moment.
- $\|\mathbf{q}\|$ is the same for every document, so it cannot change the ranking. Only $\|\mathbf{x}_d\|$ does real work, keeping long documents from winning by length alone.

Retrieval: A Worked Example

A collection of $N = 4$ documents, and the query $q = \text{"car road"}$:

document	
d_1	<i>the car is driven on the road</i>
d_2	<i>the truck is driven on the highway</i>
d_3	<i>the car is parked in the garage</i>
d_4	<i>a fast car wins the race</i>

The two query terms are weighted very differently, because they are differently common in the collection:

term	DF(f)	IDF(f)
<i>car</i>	3	0.125
<i>road</i>	1	0.602

Retrieval: A Worked Example

Take $d_1 = \text{"the car is driven on the road"}$, 7 tokens, and build $\mathbf{x}_{d_1}$ one term at a time:

term	count	TF	DF	IDF	TF-IDF $x_{d_1 f}$
<i>the</i>	2	0.286	4	0.000	0.000
<i>car</i>	1	0.143	3	0.125	0.018
<i>is</i>	1	0.143	3	0.125	0.018
<i>driven</i>	1	0.143	2	0.301	0.043
<i>on</i>	1	0.143	2	0.301	0.043
<i>road</i>	1	0.143	1	0.602	0.086

The nine remaining coordinates of $\mathbf{x}_{d_1}$, one per term of V that d_1 does not use, are all 0.

Retrieval: A Worked Example

From the weights on the previous slide, the norm of $\mathbf{x}_{d_1}$:

$$\|\mathbf{x}_{d_1}\| = \sqrt{0^2 + 0.018^2 + 0.018^2 + 0.043^2 + 0.043^2 + 0.086^2} = \sqrt{0.0117} = 0.108$$

The **query is weighted exactly like a document**: it has 2 tokens, so $\text{TF} = 0.5$ for each, times the IDF.

$$q_{car} = 0.5 \times 0.125 = 0.063 \quad q_{road} = 0.5 \times 0.602 = 0.301 \quad \|\mathbf{q}\| = \sqrt{0.063^2 + 0.301^2} = 0.307$$

We compute the dot product: $\mathbf{q} \cdot \mathbf{x}_{d_1}$, and then the cosine similarity:

$$\underbrace{0 \times 0.000}_{the} + \underbrace{0.063 \times 0.018}_{car} + \underbrace{0 \times 0.018}_{is} + \underbrace{0 \times 0.043}_{driven} + \underbrace{0 \times 0.043}_{on} + \underbrace{0.301 \times 0.086}_{road} + \underbrace{0 \times 0 + 0 \times 0 + \dots}_{9 \text{ other terms unused}} = 0.027$$

$$\text{score}(q, d_1) = \frac{0.027}{0.108 \times 0.307} = \mathbf{0.811}$$

Retrieval: Scoring and Ranking

We repeat the computation for each document.

	$\text{tfidf}(\text{car})$	$\text{tfidf}(\text{road})$	$\mathbf{q} \cdot \mathbf{x}_d$	$\ \mathbf{x}_d\ $	score	rank
d_1	0.018	0.086	0.0270	0.108	0.811	1
d_3	0.018	0	0.0011	0.151	0.024	2
d_4	0.021	0	0.0013	0.202	0.021	3
d_2	0	0	0	0.137	0.000	4

d_1 , *the car is driven on the road*, wins by a wide margin, and it is the only document containing *road*, the rarer and therefore more informative of the two query terms. d_3 and d_4 both contain *car* and are scored, but far below. d_2 shares no query term at all and scores exactly 0.

Doing This Efficiently

Scoring the query against every document works for four documents.

- For a web-scale collection, N is in the billions, and almost all of that work is wasted: **a document with no query term always scores 0.**
- In the example, d_2 , *the truck is driven on the highway*, never had any chance for a good ranking with the query *car road*.

We want a data structure that answers the question "*which documents contain this term?*" directly, without touching the rest of the collection. That structure is the **inverted index**.

It is inverted in the sense that the **document-term** matrix is read the other way round: instead of going from a document to its terms, we go from a term to its documents, i.e., we use a **term-document** matrix.

The Inverted Index

For each term in the vocabulary, store its **postings list**: the documents it occurs in, with the count needed for weighting.

To answer "*car road*": read two postings lists, take their union $\{d_1, d_3, d_4\}$, and score only those. d_2 is never looked at.

term	DF	postings list		
<i>car</i>	3	$d_1:1$	$d_3:1$	$d_4:1$
<i>road</i>	1	$d_1:1$		
<i>driven</i>	2	$d_1:1$	$d_2:1$	
<i>garage</i>	1	$d_3:1$		

- The document frequency is stored alongside the term, so **IDF** is **available without any extra pass**.
- Postings lists are kept in document order, which makes unions and intersections a linear merge.
- Sparsity helps: a term appears in very few documents, so its postings list is short.

What Sparse Retrieval Cannot Do

Ranking by cosine over TF-IDF vectors is fast, interpretable and still a strong baseline. But it matches on **exact terms**, and that is a real limit:

- **Synonymy**: a query for *car* scores 0 on a document containing only *automobile*.
 - The two words are different coordinates, and one-hot vectors of different words are orthogonal, so there is no way to know they are related.
 - Later in the course we'll give words their own vectors, learned from the contexts they appear in, so that *car* and *automobile* end up close.
- **Polysemy**: a query for *bank* retrieves river banks and financial ones alike, since one coordinate carries both senses.
- **No notion of similarity between coordinates**: the vocabulary is a set of unrelated labels.

This *vocabulary mismatch* problem is not a flaw in the ranking function. It is a flaw in the one-hot encoding **representation**.

Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. (Jan 2026 draft)

- ch. 5.2-5.5, *Modeling Word Meaning Using Lexical Co-Occurrence*: the cosine
- ch. 11, *Information Retrieval*: ad hoc retrieval (11.1), the vector space model (11.1.1), term weighting (11.1.2), a worked retrieval example (11.1.3), the inverted index (11.1.4)